

Research paper

Scalable multi-agent reinforcement learning for factory-wide dynamic scheduling in semiconductor manufacturing

Jaeyeon Jang^{a,*,}, Diego Klabjan^b, Han Liu^c, Nital S. Patel^d, Xiuqi Li^d, Balakrishnan Ananthanarayanan^d, Husam Dauod^d, Tzung-Han Juang^b^a Department of Data Science, The Catholic University of Korea, 43 Jibong-ro, Bucheon 14662, the Republic of Korea^b Department of Industrial Engineering and Management Sciences, Northwestern University, 2145 Sheridan Road, Evanston, IL 60208, USA^c Department of Computer Science, Northwestern University, 2233 Tech Drive, Evanston, IL 60208, USA^d Intel Corporation, 5000 W Chandler Blvd, Chandler, AZ 85226, USA

ARTICLE INFO

Keywords:

Dynamic scheduling
Reinforcement learning
Multi-agent systems
Semiconductor manufacturing

ABSTRACT

Real-time dynamic scheduling in modern manufacturing is highly complex due to frequent disturbances and intricate operational constraints. While reinforcement learning (RL) has shown promise, existing approaches often rely on extensive dispatching rules and struggle to scale to factory-wide settings. This study introduces a scalable multi-agent RL (MARL) framework with a leader–follower structure, enabling decentralized agents to handle sub-problems while maintaining global coordination through abstract goals. To further enhance robustness, a limited rule-based conversion algorithm is proposed to mitigate performance degradation from poor agent decisions. Our experimental results demonstrate that the proposed model outperforms the state-of-the-art deep RL-based scheduling methods in various aspects. Additionally, the proposed model provides the most robust scheduling performance to demand changes. Overall, the proposed MARL-based scheduling model presents a promising solution to the real-time scheduling problem, with potential applications in various manufacturing industries.

1. Introduction

Scheduling in semiconductor manufacturing is an exceptionally challenging task due to several complex characteristics. These include fluctuating demand, a variety of product types, intricate precedence constraints of operations, hundreds of machines spread across numerous workstations, a high frequency of re-entrant phenomena, frequent machine breakdowns and maintenance, and a vast number of possible routes for a job (Shiue et al., 2021; Kim et al., 2017; Yu et al., 2021; Chen et al., 2019). This complicated and challenging scheduling task has been represented, at a high level of abstraction, as a flexible job shop scheduling problem (FJSSP) (Gao et al., 2015; Luo, 2020) and diverse methods have been proposed to tackle the FJSSP (Fattahi et al., 2007; Amjad et al., 2018; Gao et al., 2019a).

With the increasing complexity and uncertainty of modern manufacturing systems, the FJSSP has garnered considerable attention in recent years (Gao et al., 2019a). This problem builds upon the conventional job shop scheduling problem by increasing flexibility and universality to address advanced manufacturing systems such as automobile assembly, chemical material processing, and semiconductor manufacturing (Guo et al., 2016; Gao et al., 2019b). In the FJSSP,

each operation can be processed on multiple compatible machines, each with a different processing time. As a result, the FJSSP is a formidable NP-hard problem (Kacem et al., 2002).

Many algorithms have been developed to address the FJSSP problem (Gao et al., 2015; Fattahi et al., 2007; Amjad et al., 2018; Gao et al., 2019a). However, most of them rely on an unrealistic assumption of a static scheduling environment where all the relevant information about the production environment is known in advance, and all future events are controlled as planned. The aim of these algorithms is to create the best deterministic scheduling plan without considering any future modifications. However, in complex manufacturing systems, dynamic events such as machine breakdowns, unscheduled maintenance, job insertion, and modification of order have become increasingly common (Ouelhadj and Petrovic, 2009). As a result, a deterministic schedule is not usually executed as planned, deteriorating the production efficiency significantly. In addition, factory scheduling involves many hard operational constraints due to the complexity and uncertainty of semiconductor manufacturing systems. Therefore, developing a real-time scheduling method capable of handling these constraints is of utmost importance. Recently, reinforcement learning

* Corresponding author.

E-mail address: jaeyeon.jang@catholic.ac.kr (J. Jang).<https://doi.org/10.1016/j.engappai.2025.112168>

Received 25 September 2024; Received in revised form 18 June 2025; Accepted 30 August 2025

Available online 10 September 2025

0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(RL) has achieved huge attention in this area due to its ability to learn a good policy by trial and error in a complex environment, thereby enabling real-time decision-making (Luo, 2020; Liu et al., 2020; Chang et al., 2022; Min and Kim, 2022).

Dynamic FJSSP (DFJSSP) has been extensively studied over the past decade, with dispatching rules and metaheuristics being commonly used techniques (Lou et al., 2012). Dispatching rules are popular among practitioners due to their simplicity and time efficiency. However, their solutions may not be of high quality since no single best rule can be applied to all situations (Min and Kim, 2022). In contrast, metaheuristics can provide higher quality solutions by breaking down the dynamic scheduling problem into a series of static sub-problems (Luo, 2020). Nevertheless, metaheuristics algorithms, such as genetic algorithm, tabu search, and particle swarm optimization, may suffer from significant computational cost due to their huge search spaces (Kundakci and Kulak, 2016; Ning et al., 2016). Thus, their practicality is questionable.

With the rapid advancement of artificial intelligence, deep RL (DRL) has emerged as an important approach in real-time scheduling due to its practicality in adapting to numerous dynamic events in real-time (Cunha et al., 2020). Despite its performance improvement over dispatching rules and metaheuristics, existing DRL-based methods have several limitations. Firstly, most of them heavily rely on dispatching rules. Secondly, prior work does not consider hard operational constraints, which are strict conditions that must be satisfied for a schedule to be feasible, in dynamic environments which is crucial for a factory-wide production environment.

The factory-wide DFJSSP presents a challenge for obtaining optimal or near-optimal solutions due to its complex nature and numerous hard operational constraints. In this study, ‘factory-wide’ refers to the comprehensive scheduling of all stages and machines across the entire production line in a complex manufacturing system. Unlike classical FJSSP settings that typically model only a portion of a production process, the factory-wide problem encompasses the entire production pipeline. This includes coordinating hundreds of machines across multiple workstations, handling diverse product types, and responding to dynamically occurring events such as machine breakdowns or demand fluctuations. As a result, the problem becomes substantially larger in scale, more constrained, and highly sensitive to operational disturbances. Even the introduction of DRL cannot address this challenge as the entire search space becomes impractical (Subramanian et al., 2016). To overcome this limitation, we introduce the concept of a multi-agent RL (MARL) approach (Zhang et al., 2021) that decomposes the problem into a set of operation-specific sub-problems. In this concept, achieving team goals requires effective coordination among all agents (Foerster et al., 2018; Rashid et al., 2018; Yang et al., 2020). However, in the context of a factory-wide production environment, guiding multiple agents with commonly used shared team reward (Tan, 1993) is not effective due to the complex hierarchy and intricate interconnections of numerous sequential operations. Therefore, instead of a shared reward, we use operation-wise rewards that allow each agent to focus on its assigned operation, thus significantly reducing the search space. However, cooperation may not be achieved through this approach, even if each agent performs well in its assigned operation. To address this challenge, we introduce a new outer agent proposed in Jang et al. (2023), called the leader, which coordinates the agents assigned to each operation, which we call followers, by providing an abstract goal for communication.

In a real factory production environment, converting from one product type to another requires a significant operational cost. Therefore, strict constraints are imposed to ensure that conversions are carefully executed. Since there is limited flexibility in the conversion process, even one bad decision made by a follower can have catastrophic consequences. For instance, machines can unexpectedly become idle, resulting in a significant loss of production capacity over a long period of time. However, it is impossible to guarantee that RL agents will not

make any errors. Therefore, in situations where a follower’s bad decision can significantly impact performance, we implement a rule-based conversion algorithm. This allows us to apply a set of predetermined rules that supersede the follower’s decision-making process. In this way, we can prevent potential performance deterioration that may occur due to a follower’s decision.

In summary, the factory-wide DFJSSP for semiconductor manufacturing is a significant generalization of the classical FJSSP in both scale and complexity. While the classical FJSSP focuses on scheduling a limited number of operations with flexible machine assignments under static conditions, the factory-wide setting incorporates the following additional challenges.

- Massive scale: Hundreds of machines, diverse product types, and numerous operations spread across multiple workstations must be considered simultaneously, making the action and state spaces significantly larger.
- Dynamic events: Unlike classical FJSSP, factory-wide scheduling must handle real-time disturbances such as machine breakdowns, unscheduled maintenance, and variable demand patterns.
- Hard operational constraints: These include strict product changeover constraints, conversion time limits per shift, precedence constraints, and resource constraints that must be satisfied to ensure feasible schedules.
- High penalty for suboptimal decisions: A single poor decision (e.g., inappropriate product conversion) can cause machines to remain idle for extended periods, leading to a substantial production capacity loss.

These factors make factory-wide scheduling not only more complex but also less tolerant to error. To address these challenges, our proposed method adopts a scalable MARL framework with a leader–follower architecture, operation-wise rewards, and a rule-based conversion mechanism.

We benchmarked the proposed model by comparing it with the state-of-the-art DRL-based scheduling models utilizing dispatching rules in real factory environments. Experimental results show that existing DRL-based models cannot effectively learn a scheduling strategy in complex real factory environments. This reveals that a combination of predefined dispatching rules is insufficient. In contrast, our proposed model achieved significant performance improvement during training, resulting in much better scheduling strategies. Furthermore, our proposed model was found to be more robust to increasing demand, as its performance deterioration was relatively small. Specifically, compared to the existing models in the most challenging situation with high demand, the proposed model reduces tardiness by 10.4% and improves the completion rate by 31.4% on average. Our main contributions are as follows.

- An RL-based scheduling model that is not based on human-made dispatching rules is proposed.
- An MARL concept based on leader–follower that utilizes abstract goals for cooperation among followers is successfully introduced to obtain scalability for factory-wide dynamic scheduling problems.
- A rule-based conversion algorithm is developed and incorporated to prevent catastrophic loss of production capacity.

While the methodology is based on Jang et al. (2023) herein we modify it by incorporating features specific to real-world factory environments, including scheduled maintenance and unexpected machine breakdowns. Additionally, we omit the concept of synthetic rewards present in Jang et al. (2023) and instead developed an operation-wise rewarding strategy alongside a rule-based conversion algorithm to prevent catastrophic loss of production capacity. The remainder of this paper is organized as follows. In Section 2, related works on RL-based dynamic scheduling are overviewed. Section 3 formulates

factory-wide DFJSSP, the target scheduling problem of this paper. In Section 4, we detail the proposed model, incorporating the leader-follower MARL algorithm from Jang et al. (2023), which we integrate into our scheduling model. This approach is utilized to coordinate multiple agents in factory-wide scheduling, a problem characterized by a directed acyclic graph (DAG). In Section 5, the proposed model is thoroughly benchmarked with various baselines including several state-of-the-art models. Finally, Section 6 concludes this study by discussing some limitations and future research directions.

2. Related works

2.1. Heuristic-based scheduling approaches

Early research on dynamic scheduling problems has predominantly relied on rule-based heuristics and metaheuristic algorithms (Ouelhadj and Petrovic, 2009). These approaches are known for their simplicity and practical effectiveness in simple factory environments. Common dispatching rules, such as shortest processing time (SPT), earliest due date, and least slack time, offer quick decisions based on local system states, but often fall short in complex and dynamic settings due to their myopic nature.

To overcome the limitations of simple heuristics, numerous studies have explored metaheuristics such as simulated annealing (Dehghan-Sanej et al., 2021), genetic algorithm (Li and Gao, 2020), particle swarm optimization (Wang et al., 2019), and ant colony optimization (Zhang et al., 2020). These techniques allow for broader search capabilities across the solution space and are particularly effective in static or moderately dynamic environments. Genetic programming (GP), in particular, has been widely used to accommodate dispatching rules, demonstrating adaptability across various dynamic scheduling instances (Zhang et al., 2024; Xu et al., 2024). Recent studies also emphasize the importance of feature selection and representation in GP-based approaches for improved rule generalization (Nguyen et al., 2013; Branke et al., 2015; Shady et al., 2022).

While these methods can achieve high-quality schedules, they generally lack the real-time adaptability required in highly dynamic environments. Additionally, the handcrafted nature of rule sets and parameter tuning in metaheuristics can hinder scalability and robustness in large-scale manufacturing systems.

2.2. RL-based scheduling approaches

RL has proven to be successful in various fields, including robot control (Cui et al., 2017), manufacturing (Hein et al., 2018), and gaming (Rashid et al., 2018) in the past few decades. As a result, many researchers in the scheduling field have become interested in this learning technique (Luo, 2020; Liu et al., 2020; Chang et al., 2022; Min and Kim, 2022). They have adopted the concept of training adaptable agents to choose the best action (usually dispatching rules) at decision points through numerous trial and error. More specifically, dynamic scheduling problems have traditionally been approached as Markov decision processes (MDPs), which are solved using classical RL algorithms such as Q-learning and SARSA (Sutton and Barto, 2018). For example, Aydin and Öztelme (2000) applied the Q-learning algorithm to train an agent that selects the most appropriate rule among three popular dispatching rules: the SPT rule, the cost over time rule, and the critical ratio rule. Their goal was to determine the next job in real-time, with the aim of minimizing mean tardiness. Similarly, Wei and Zhao (2005) introduced an agent trained using Q-learning to select the best rule at each decision point after designing several composite dispatching rules. In Wang and Usher (2005), Q-learning was used to select the best dispatching rule among the FIFO rule, the earliest due date rule, and the SPT rule based on the number of waiting jobs and the estimated total lateness. Xili et al. (2010) used Q-learning to produce a single composite rule by subsequently combining several dispatching

rules based on a weighted aggregation method. Finally, Shahrabi et al. (2017) utilized Q-learning to obtain optimal parameters for the variable neighborhood search algorithm, a heuristic method that searches for the best schedules in dynamic job shop problems with unequal job arrival times and machine failures. While conventional RL-based scheduling methods have made significant progress, the latest production systems have presented more scheduling challenges, requiring exploration of larger state spaces (Luo et al., 2021) and making the conventional RL approaches less effective.

In advanced production systems, there are almost infinite numbers of different states, making it difficult for conventional RL algorithms to obtain a good solution within a limited time budget, which raises questions about their practicality. However, with the emergence of deep learning, DRL has made breakthroughs in real-time scheduling with large and continuous state spaces by using deep neural networks (DNNs) to estimate the value function and/or policy (Mnih et al., 2015; Lillicrap et al., 2016). Consequently, recent developments in the field of scheduling have largely been achieved through DRL. For instance, Waschneck et al. (2018) applied the concept of MARL while using a deep Q-network (DQN) so that agents can determine the best dispatching rules for a work center to maximize machine utilization. Hu et al. (2020) proposed a graph convolutional network-based DQN to address route flexibility and stochastic arrivals of products in flexible manufacturing systems. Liu et al. (2020) introduced an actor-critic architecture while applying a parallel training method that combines asynchronous updates and deep deterministic policy gradient (DDPG). In this work, multiple agents were optimized to determine the best rule among several simple dispatching rules for each machine. Similarly, Gui et al. (2023) employed DDPG to learn a continuous-weighted composite scheduling rule, showing superior performance over Q-learning and DQN in minimizing tardiness. Min and Kim (2022) used a DDPG algorithm to dynamically tune the parameters of the apparent tardiness cost dispatching rule, which is known to yield schedules that minimize the total weighted tardiness (Vepsäläinen and Morton, 1987). Luo (2020) designed six composite dispatching rules and proposed double DQN (DDQN) to determine the best rule among the designed dispatching rules at every rescheduling point to minimize total tardiness. In the following work, Luo et al. (2021) extended Luo's model to a two-hierarchy deep Q-network by introducing a higher-level DDQN that determines the optimization goal. Chang et al. (2022) enhanced DDQN by proposing a soft ϵ -greedy behavior policy to balance exploration and exploitation but still require the trained agent to perform the same task of choosing a dispatching rule from a set of rules. Wesendrup and Hellingrath (2023) demonstrated the effectiveness of proximal policy optimization (PPO) in jointly optimizing post-prognostics production, spare parts, and maintenance planning under uncertainty. Park et al. (2021) proposed a scheduling agent based on graph neural networks for job-shop scheduling, where PPO was used to train the agent to select optimal scheduling actions. Similarly, Wang et al. (2021) employed PPO to train an agent for deriving effective scheduling policies in dynamic job-shop environments. Gui et al. (2024) proposed a multi-agent DDPG (MADDPG)-based scheduling method for a self-organizing manufacturing system, encapsulating heterogeneous equipment as agents and demonstrating significant gains in coordination and scheduling robustness. These recent advances underscore the versatility and growing relevance of both advanced RL algorithms and MARL approaches in handling large-scale and dynamic scheduling problems, reinforcing the direction of this work.

While the existing methods have been effective for solving dynamic scheduling problems, there are weak points. Specifically, they assume that there are no restrictions on job selection and that numerous product type conversions can be executed. However, in reality, conversion is significantly limited to minimize operational costs and maximize machine utilization, while also considering operational constraints within a factory. As a result, these existing scheduling methods cannot be applied to real-world factory environments or may significantly impair scheduling performance. Thus, this study proposes a MARL based scheduling algorithm that specifically targets real-world factory environments.

2.3. Leader–follower MARL

Recently, several studies have proposed deep RL-based leader–follower MARL frameworks to address more general multi-agent problems. For example, [Shu and Tian \(2019\)](#) introduced a hierarchical architecture in which a leader agent assigns sub-tasks to followers with different preferences and capabilities. However, their approach assumes that follower agents are rule-based and thus lacks adaptability in dynamic environments. [Yu et al. \(2020\)](#) proposed an enhanced leader–follower MARL algorithm by incorporating a sequential decision module, leveraging the temporal dependency between goals and rewards to improve coordination efficiency. [Jiang and Lu \(2021\)](#) presented a novel approach called “emergence of individuality”, which uses a probabilistic classifier to generate intrinsic reward signals based on agent-specific observation distributions, encouraging diverse behavior. While these methods contribute to the broader understanding of hierarchical MARL, they primarily focus on generic cooperative or non-cooperative settings and overlook the structured operational dependencies and constraints found in real-world scheduling problems. In contrast, our proposed model applies a leader–follower MARL framework specifically designed for factory-wide scheduling. [Zhang et al. \(2022\)](#) introduced a leader-following MARL paradigm in which the leader broadcasts its chosen action, and followers adapt their value functions through a hypernetwork conditioned on this broadcast. This design not only enhances cooperation but also allows for explicit modeling of leader influence via information gain. [Xiao et al. \(2023\)](#) proposed a homogeneous graph neural network based MADDPG algorithm that improves leader–follower flocking by incorporating graph structure and homogeneous policy sharing to enhance stability and coordination; however, it requires agents to be homogeneous, limiting its applicability to more diverse settings.

These recent advancements highlight a transition in leader–follower MARL from rigid sub-task allocation to dynamic, learning-based coordination mechanisms. Our work builds upon this concept by allowing both leader and followers to learn policies in a fully decentralized manner, while maintaining asynchronous decision structures and modeling rich dependencies through communication-aware learning.

3. Problem statement

The factory-wide DFJSSP for semiconductor manufacturing assumes that jobs with the same product type have identical unit processing times for an operation. We also assume earlier arriving jobs are processed first when multiple jobs with the same product type are waiting to be processed. When a machine undergoes a change in operation mode or product type, a conversion time depending on the previous operation/product type and the new one is needed.

We describe factory-wide DFJSSP as a planning problem consisting of a period of N shift. Given a product type p , we denote K_p to be the number of lots of demand and J_p to be the number of required operations. To produce a product p , we need to sequentially conduct a sequence of operations $\{O_{p,j} | j = 1, 2, \dots, J_p\}$ where $O_{p,j}$ is the j th operation of p . We can perform each operation $O_{p,j}$ on a set of compatible machines $\mathbf{M}_{p,j} \subseteq \mathbf{M}$, where $\mathbf{M}$ is the set of all machines. In general, most machines have the ability to produce multiple type of products and perform multiple operations. However, changing operations or product types requires conversion time. To this end, we denote CO_{p_0, o_0, p_1, o_1} to be the time required to convert from operation o_0 for product type p_0 to operation o_1 for product type p_1 . To better reflect a real-world factory environment, we also impose a conversion time threshold TH on all machines for each shift. This is necessary since excessive conversion time can have negative impacts on machine utilization and productivity. Specifically, in a given shift of length S , the total conversion time for each machine cannot exceed TH . The time required to process the k th lot of product type p on operation $O_{p,j}$ is the

product of the number of units in the lot $U_{p,k}$ and the unit processing time $PR_{p,j}$.

In real-world factory environments, both scheduled maintenance to prevent machine breakdowns and unscheduled maintenance due to unexpected breakdowns must be considered. For machine l , $SM_{l,t}$ is defined as 1 if l is under scheduled maintenance at time t , and 0 otherwise. Additionally, Ω_t represents the set of machines undergoing maintenance due to unexpected breakdowns, which is really a random event. For simplicity, we assume that unscheduled maintenance begins only after the current lot is processed.

Given the provided setting, we list the decision variables in factory-wide DFJSSP.

- $X_{p,j,k,l,t} = \begin{cases} 1, & \text{if } O_{p,j} \text{ is assigned on } l \in \mathbf{M}_{p,j} \\ & \text{for the } k\text{th lot of product } p \text{ at } t \\ 0, & \text{otherwise} \end{cases}$
- $C_{p,j,k}$: Completion time of $O_{p,j}$ for the k th lot of product type p
- $PT_{l,t}$: product type setup of $l \in \mathbf{M}$ at t
- $OP_{l,t}$: operation setup of $l \in \mathbf{M}$ at t
- $Q_{l,t} = \begin{cases} 1, & \text{if } l \in \mathbf{M} \text{ is not idle at } t \\ 0, & \text{otherwise} \end{cases}$

Variable $X_{p,j,k,l,t}$ determines which machine performs $O_{p,j}$ for the k th lot of product type p , while $Q_{l,t}$ characterizes the status of M_l . Specifically, $Q_{l,t}$ is set to 1 if l is currently processing, undergoing a setup change, or undergoing scheduled or unscheduled maintenance; otherwise, it is set to 0, as detailed in (4). Whenever there is an idle machine, a decision should be promptly made for that machine. For example, if $Q_{l,t} = 0$ for an $l \in \mathbf{M}_{p,j}$, and $(k-1)$ -th lot of p has already been processed in $O_{p,j}$, we can set $X_{p,j,k,l,t} = 1$ to initiate k th lot of p using l for $O_{p,j}$. Conversely, if no machines are available at time t , the decision is set by fixing $X_{p,j,k,l,t} = 0$. Formally,

$$X_{p,j,k,l,t} \leq (1 - Q_{l,t}) \mathbf{1}_{\mathbf{M}_{p,j}}(l) \sum_{l \in \mathbf{M}_{p,j}} \sum_{\tau=1}^{t-1} X_{p,j,k-1,l,\tau}. \quad (1)$$

Additionally, let $D_{p,k}$ be the due time of lot k of product p . Our goal is to identify decision variables that either minimize the number of delayed jobs, equivalently maximize the completion rate, or alternatively minimize total tardiness, within the planning horizon of N shifts, where each shift comprises of S decision points. In this paper, for simplicity, we assume that lots within the same product type contain nearly identical unit counts. Consequently, our objective function, formalized as (2) or (3), does not account for variations in unit numbers among lots of the same product type. In the following, the expectation is with respect to unscheduled maintenance captured by random events Ω_t . Equations and constraints (1) and (4)–(12) hold for any realization of Ω_t . To minimize the number of delayed jobs, the objective function is given by:

$$\text{Minimize } \mathbb{E} \left[\sum_p \sum_{k=1}^{K_p} H_{p,J_p,k} \right] \quad (2)$$

$$\text{where } H_{p,J_p,k} = \begin{cases} 1, & \text{if } D_{p,k} < C_{p,J_p,k} \\ 0, & \text{otherwise} \end{cases}$$

or, to minimize total tardiness:

$$\text{Minimize } \mathbb{E} \left[\sum_p \sum_{k=1}^{K_p} (C_{p,J_p,k} - D_{p,k}) H_{p,J_p,k} \right], \quad (3)$$

subject to

$$Q_{l,t} = \mathbf{1}_{\Omega_t}(l) SM_{l,t} \sum_p \sum_{k=1}^{K_p} \sum_{j=1}^{J_p} \mathbf{1}_{\mathbf{M}_{p,j}}(l) \left(\sum_{\tau=t-PR_{p,j}U_{p,k}}^t Z_{p,j,l,\tau} X_{p,j,k,l,\tau} \right)$$

$$+ \sum_{\tau=t-(P_{p,j}U_{p,k}+CO_{PT_{l,t},OP_{l,t},p,O_{p,j}})}^t (1 - Z_{p,j,l,\tau})X_{p,j,k,l,\tau}\forall l, t, \quad (4)$$

$$\text{where } Z_{p,j,l,t} = \begin{cases} 1, & \text{if } PT_{l,t} = p \text{ and } OP_{l,t} = O_{p,j} \\ 0, & \text{otherwise} \end{cases}$$

$$Q_{l,t} \leq 1\forall l, t, \quad (5)$$

$$PT_{l,t} = \begin{cases} p, & \text{if } X_{p,j,k,l,t} = 1 \\ PT_{l,t-1}, & \text{otherwise} \end{cases} \quad (6)$$

$$OP_{l,t} = \begin{cases} O_{p,j}, & \text{if } X_{p,j,k,l,t} = 1 \\ OP_{l,t-1}, & \text{otherwise} \end{cases} \quad (7)$$

$$C_{p,j,k} = \sum_{l \in M_{p,j}} \sum_{t=1}^{NS} (Z_{p,j,l,t}X_{p,j,k,l,t}(t + PR_{p,j}U_{p,k}) + (1 - Z_{p,j,l,t})X_{p,j,k,l,t}(t + CO_{PT_{l,t},OP_{l,t},p,O_{p,j}} + P_{p,j}U_{p,k}))\forall p, j, k, \quad (8)$$

$$C_{p,j+1,k} \geq C_{p,j,k} + PR_{p,j+1}U_{p,k} + \sum_{l \in M_{p,j+1}} \sum_{t=C_{p,j,k}}^{NS} (1 - Z_{p,j+1,l,t})X_{p,j+1,k,l,t}CO_{PT_{l,t},OP_{l,t},p,O_{p,j+1}}\forall p, j, k, \quad (9)$$

$$C_{p,j,k} - PR_{p,j}U_{p,k} \geq C_{p,j,k-1} - PR_{p,j}U_{p,k-1}\forall p, j, k, \quad (10)$$

$$\sum_{t=1}^{NS} \sum_{l \in M_{p,j}} X_{p,j,k,l,t} \leq 1\forall p, j, k, \quad (11)$$

$$\left(TH - \sum_p \sum_{k=1}^{K_p} \sum_{j=1}^{J_p} \sum_{\tau=(n-1)S+1}^t (1 - Z_{p,j,l,\tau})X_{p,j,k,l,\tau}CO_{PT_{l,t},OP_{l,t},p,O_{p,j}} \right) \sum_p \sum_{k=1}^{K_p} \sum_{j=1}^{J_p} \sum_{\tau=t}^{nS} (1 - Z_{p,j,l,\tau})X_{p,j,k,l,\tau} \geq 0\forall l, t \in \{(n-1)S+1, \dots, nS\}, n \in \{1, 2, \dots, N\}. \quad (12)$$

Eqs. (4) and (5) make sure that a machine cannot process multiple jobs simultaneously. Additionally, (6) and (7) ensure that a machine only changes its setup when necessary, such as when a new product type or operation is assigned. If a setup change occurs, the machine requires both processing time and conversion time to complete the job, but only processing time is required if there is no setup change, as stipulated in constraint (8). Here, NS represents the total number of decision points occurring within an episode, which spans the planning horizon of N shifts. To ensure that jobs are executed in the proper sequence, (9) guarantees that an operation can only be executed once the previous operation for a job is completed. The first-in, first-out (FIFO) rule is applied to lots of the same product type that are waiting for the same operation, as described in constraint (10). Eq. (11) guarantees that a lot can only be assigned to one machine once for an operation. If the lot requires reentry into an operation, that reentry is considered a separate operation in sequence $\{O_{p,j}|j = 1, 2, \dots, J_p\}$. In this way, we can incorporate the reentry feature of factory scheduling into our model. Finally, constraint (12) ensures that a conversion cannot be executed if the cumulative conversion time exceeds TH within a single shift.

4. Scheduling model for factory-wide DFJSSP

In this section, we introduce a scheduling model based on MARL designed to effectively address the factory-wide DFJSSP. The proposed model features two types of agents: followers and a leader. We begin by presenting the formulation for the followers and the leader, detailing their respective state and action representations along with the reward mechanisms. Next, we describe the simulation environment used for model training and training algorithm employed to train the agents within our scheduling model. Finally, to minimize production capacity loss by preventing catastrophic mistakes by agents, we introduce a rule-based conversion algorithm.

4.1. Follower model

In our model, coordination among followers (i.e., lower-level agents) is achieved through a centralized leader that observes the global state and assigns abstract goal vectors to each follower at the start of every shift. Each follower is responsible for a single operation and independently makes machine-level scheduling decisions based on its local state and the provided goal vector. This design reduces the complexity of joint action spaces while allowing the system to remain globally coordinated. The leader's influence is indirect but effective—by modifying the goals, it dynamically adjusts follower behaviors to improve overall scheduling outcomes.

As depicted in Fig. 1(a), each follower observes both the states of the machines assigned to that operation and operation-specific attributes, such as processing time $\{PR_{p,j}\}_{p,j}$, work-in-process (WIP), estimated from $\{PR_{p,j}, X_{p,j,k,l,t_0}\}_{p,j,k,l,t_0 \leq t}$, and demand inferred from due dates $\{D_{p,k}\}_{p,k,l,t_0 \leq t}$. This design ensures that the follower has access to all relevant information required for effective decision-making at the operation level. Specifically, the machine state elements, such as the current setup ($PT_{l,t}$ and $OP_{l,t}$) and status ($Q_{l,t}$), are encoded using one-hot encoding to represent the current product and operation for a machine l at time t . Both scheduled and unscheduled maintenance are represented identically by setting $Q_{l,t} = 1$. The cumulative conversion time for the current shift and the next available time are also tracked to predict future availability of the machine and products. The cumulative conversion time of machine l at t is calculated based on $\{PT_{l,t_0}, OP_{l,t_0}, CO_{p,o,PT_{l,t_0},OP_{l,t_0}}|\forall o, p, t_0 \leq t\}$, and the next available time is determined from $\{U_{p,k}, PR_{p,j}, X_{p,j,k,l,t_0}|\forall p, j, k, t_0 \leq t\}$. If scheduled maintenance is in progress, the next available time is set to the end of the maintenance period. Each agent maintains these machine state elements for all machines pertinent to the target operation. Additionally, WIPs awaiting the operation and both delayed and future demand lots for all available product types in the target operation are encapsulated in the state vector. These can be readily derived from $\{D_{p,k}, C_{p,j,k}, X_{p,j,k,l,t_0}|\forall j, k, l, t_0 \leq t\}$ for product type p . The underlying formulas are given in (4)–(12). Additionally, at the beginning of each shift, the leader distributes abstract goal vectors for each follower, and the abstract goal is incorporated into the follower's state to direct them towards achieving a higher reward. Details are provided later.

Product setup changes occur intermittently in real-world factories. Consequently, the followers must make crucial decisions regarding conversion for all machines assigned to the operation, as illustrated in Fig. 1(b). To accomplish this, followers produce action vectors $a_{n,t}^o$ for all machines within the target operation o at t in the n th shift. If multiple machines belong to operation o , the follower o 's action is the concatenation of actions for all member machines. That is, $a_{n,t}^o = (a_{n,t}^{o,l}|l \in \text{available machines for operation } o)$, where $a_{n,t}^{o,l}$ is the action for the available machine l in operation o . Each action vector $a_{n,t}^{o,l}$, a one-hot encoded vector, represents two factors for a machine: whether or not to convert and the next product type. In Fig. 1(b), $p_{o,i}$ is i th available product type in operation o and np_o is the number of available product types in operation o . If the first entry is activated, $p_{o,1}$ is selected as the

next product type. Otherwise, there is no change made to the product setup. Let $\psi_{l,t}^{p_{o,i},1}$ be set to 1 if $p_{o,i}$ is chosen as the next product setup for machine l at t via the activation of $(a_{n,t}^{o,i})_{p_{o,i},1}$. Conversely, $\psi_{l,t}^{p_{o,i},0}$ should be set to 1 either if $(a_{n,t}^{o,i})_{p_{o,i},0}$ is activated or if another product is chosen as the setup. By default, both $\psi_{l,t}^{p_{o,i},1}$ and $\psi_{l,t}^{p_{o,i},0}$ are initialized to 0. Then, for any machine l , we have

$$\psi_{l,t}^{p_{o,i},0} + \psi_{l,t}^{p_{o,i},1} = 1 \quad (13)$$

and

$$\sum_{i=1}^{np_o} \psi_{l,t}^{p_{o,i},1} \leq 1. \quad (14)$$

The next product type is only relevant when a conversion is triggered, and conversions can only occur if there is WIP of the selected product in the operation. Let us assume that $p_{o,i}$ is selected and a conversion is initiated by an action vector for machine l at time t , corresponding to operation $o = O_{p,j}$. If there is a waiting lot k , then $X_{p_{o,i},j,k,l,t+1}$ is set to 1; otherwise, it remains 0. Conversely, if no conversion is triggered but there is an upcoming lot k for the current setup $p_{o,i}$ of machine l at time t , then $X_{p_{o,i},j,k,l,t+1}$ is also set to 1.

It is not always necessary for all followers to take action since there may be a lack of available machines, as depicted in Fig. 2(b). In this situation, only the followers who have access to at least one available machine are able to produce actions. The action of the follower o at t in the n th shift, $a_{n,t}^o$, is determined according to

$$a_{n,t}^o \sim \pi^o \left(\cdot \mid s_{n,t}^o, g_n^o \right), \quad (15)$$

where π^o represents the policy of the follower responsible for operation o , while g_n^o denotes the goal vector for the follower during the n th shift.

In the MARL-SR algorithm (Jang et al., 2023), synthetic rewards are utilized to account for each agent's contribution to the overall team reward. However, given that the demand for each product type is specified for each shift, we apply more straightforward reward calculation methods. One approach penalizes followers based on the number of delayed lots. Specifically, if we define the reward using the number of delayed lots, the reward r_n^o for the operation o , given after n th shift, is computed as:

$$r_n^o = - \sum_{i=1}^{np_o} \sum_{k=1}^{K_{p_{o,i}}} H_{p_{o,i},j,k}, \quad (16)$$

$$\text{where } H_{p_{o,i},j,k} = \begin{cases} 1, & \text{if } D_{p_{o,i},k} < C_{p_{o,i},j,k} \text{ and } D_{p_{o,i},k} \leq nS \\ 0, & \text{otherwise.} \end{cases}$$

Alternatively, tardiness can be used as the reward signal. In this case, the reward is defined as

$$r_n^o = - \sum_{i=1}^{np_o} \sum_{k=1}^{K_{p_{o,i}}} \max(C_{p_{o,i},j,k} - D_{p_{o,i},k}, 0) H_{p_{o,i},j,k}. \quad (17)$$

In summary, each follower agent receives a reward after every shift based on the number of delayed lots or tardiness within its assigned operation. This operation-wise reward design ensures that each agent focuses on optimizing its own localized scheduling sub-problem.

The training environment is quite challenging, with the majority of actions being trivial and sparse rewards being provided. To supplement potentially insufficient training and prevent followers from generating inadequate actions that can result in catastrophic production capacity losses, we introduce a rule-based conversion algorithm in Section 4.4.

4.2. Leader model

Factory-wide DFJSSP requires a high level of coordination between multiple entities, such as jobs, machines, and operations. Traditional single agent based RL methods do not perform well due to the exponential growth of the joint action space as the number of entities

increases (Rashid et al., 2018). To address this challenge, MARL has been proposed in which each agent aims to obtain the best policy for an entity or sub-problem (Zhang et al., 2021). This decentralization is based on the premise that achieving individual goals assigned to each agent is easier than attaining team success. However, individual success is meaningless unless team goals are achieved. Therefore, it is crucial to maximize team reward through cooperation while pursuing individual goals in MARL.

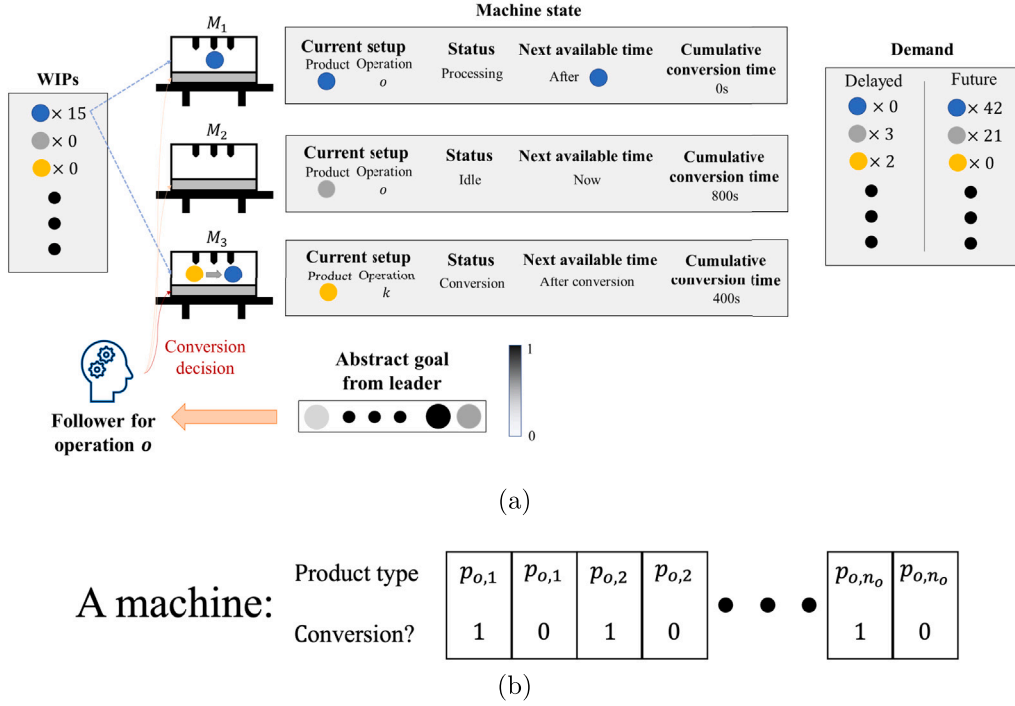
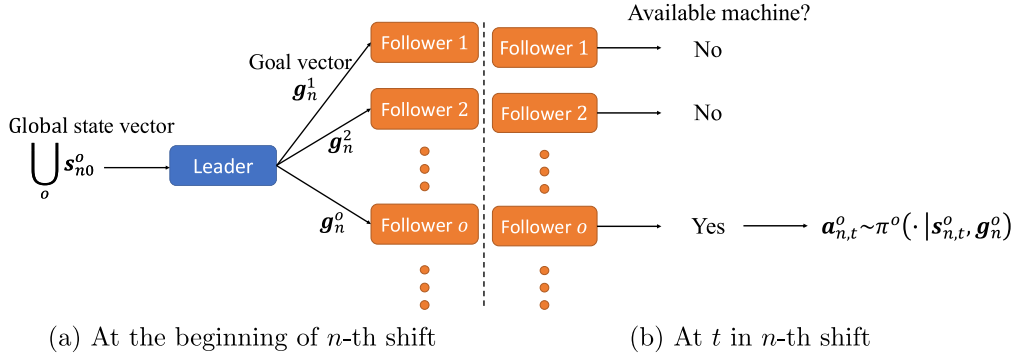
One approach of MARL is to train decentralized policies in a centralized manner, as this approach can offer valuable state information and eliminate communication barriers among agents (Rashid et al., 2018; Yang et al., 2020; Oliehoek and Vlassis, 2007; Kraemer and Banerjee, 2016; Lowe et al., 2017). Despite the increasing interest in centralized training with decentralized execution in the RL community, this method is not viable for use in advanced production systems due to the high synchronization demand between agents. Especially, factory-wide scheduling involves numerous precedence constraints that can be represented as a DAG. Consequently, this scheduling problem can be modeled as a multi-agent problem with DAG constraints, where each agent handles a specific operation. Essentially, every agent aims to maximize team rewards, considering the complex interrelationships between agents. A significant challenge is that rewards can only be ascertained after multiple agents have made a series of sequential decisions over an extended period. This delay in reward distribution complicates an agent's ability to assess its contribution to team rewards at each time step, thus impeding effective learning from the perspective of the individual agent. To overcome these challenges, we use the concept of the leader proposed in Jang et al. (2023). This outer agent aims to coordinate the operation-specific agents towards achieving high team rewards by setting long-term, agent-specific goals.

The leader generates goals by taking into account all relevant factors, including the states of operations, and distributes them to each follower at the beginning of every shift. These goals are expressed in an abstract form as real numbers in $[0, 1]$ and serve as communication channels between the leader and followers. The leader's rewards are based on the followers' achievements at the end of each shift. Therefore, the leader needs to create goal vectors that contain meaningful messages to lead the followers towards higher team rewards by observing the global environment information. The team rewards are also given to the leader. The role of the leader is illustrated in Fig. 2(a). At the beginning of each shift, the leader receives the global state vector $\cup_o s_{n,0}^o$, where $s_{n,0}^o$ is the initial state vector of operation o in the n th shift. The leader utilizes the global state vector to generate individualized goals for each follower, which are then used to address specific operations. The leader's policy π^L is defined as

$$(g_n^o \mid o \in \text{all operations}) \sim \pi^L \left(\cdot \mid \bigcup_o s_{n,0}^o \right). \quad (18)$$

This approach makes it much easier to handle complex factory-wide scheduling problems, as they are decomposed into a set of weakly connected sub-scheduling problems and the problem of managing them.

While the leader has access to comprehensive information about the team's operations, followers only possess partial information. Nonetheless, they can infer the overall state of the system, including the state of the other operations and the coordination strategy of the leader, from abstract goals. Therefore, establishing effective communication channels is crucial for attaining high team rewards, utilizing goal vectors and team rewards. Unlike shared reward MARL methods, our design decouples the learning process across agents but uses the leader as a bridge to enable coordination. Although each follower observes only partial information, they can adjust their decisions in response to these goals, which serve as a compact communication signal. This goal-based communication forms the backbone of the underlying coordination in our approach.

Fig. 1. Illustration of (a) state and (b) action of the follower for operation o .Fig. 2. Overview of our MARL scheduling algorithm based on (a) leader and (b) followers. Here, π^o is the incumbent policy of the follower.

The primary objective of scheduling in this study is to maximize completion rate, and the reward for maximizing completion rate can take the form of the negative value of the number of lots or tardiness as introduced in Section 3. In addition, when creating schedules, it is assumed that the demand for each product type is given per shift. Thus, we calculate the penalty based on delayed lots in each operation after each shift. Then, a follower receives the sum of the penalties for all processable product types in the assigned operation as the reward after each shift based on (16). The leader, in contrast, receives a global team-level reward after each shift n . If the number of delayed lots is used as the reward signal, the leader's reward after the n th shift is defined as:

$$r_n^l = - \sum_p \sum_{k=1}^{K_p} H_{p,J_p,k}, \quad (19)$$

where $H_{p,J_p,k} = \begin{cases} 1, & \text{if } D_{p,k} < C_{p,J_p,k} \text{ and } D_{p,k} \leq nS \\ 0, & \text{otherwise,} \end{cases}$

whereas if tardiness is used, the reward is defined as:

$$r_n^l = - \sum_p \sum_{k=1}^{K_p} \max(C_{p,J_p,k} - D_{p,k}) H_{p,J_p,k}. \quad (20)$$

These rewards aggregate the penalties across all product lots and operations. By observing the overall system status at the beginning of each shift, the leader learns to generate goal vectors that indirectly encourage followers to align their local objectives with global performance.

4.3. Environment and training

In this section, we describe the simulation environment used to train RL agents, as summarized in Algorithm 1. Initially, we configure the production environment details, including the planning horizon (NS), product types (p), sequences of operations for each product type ($O_{p,j} \forall p, j \in 1, 2, \dots, J_p$), sets of compatible machines ($M_{p,j} \forall p, j \in 1, 2, \dots, J_p$), unit processing times ($PR_{p,j} \forall p, j \in 1, 2, \dots, J_p$), scheduled maintenance details ($SM_{l,t} \forall l, t$), and the conversion time threshold (TH). Each episode involves generating production plans for each shift over a predefined duration, detailing the required number of lots for each product type based on the incumbent policy. For each episode, it is assumed that demand information, specifying the required number of lots for each product type per shift, is provided. The objective of the RL agents in each episode is to meet this demand without

delay. This demand information serves as the basis for reward calculation. Historical data contains only few demand observations; definitely insufficient for RL training. For this reason, we randomly generate demand for each episode with the underlying distribution calibrated from historical data, ensuring it accurately reflects the actual factory environment. We consider various factors such as the distribution of demand by product type, along with a range of operational constraints and conditions – including precedence constraints, processing times, and conversion times by product type – all based on real production datasets from a high-volume packaging and test factory. Furthermore, the initial settings of the factory environment, including the status of machines and WIP, are configured at the beginning of each episode. These initial settings lay the foundation for generating schedules based on the policies of the leader and the followers.

As is typical in RL, given an incumbent policy, at the beginning of the first shift of each episode, the leader creates and distributes goal vectors, as defined in (18), which guide the followers towards achieving high team rewards. At each decision point, machine statuses are updated to ‘unavailable’ based on both scheduled ($SM_{l,t} \forall l$) and unscheduled maintenance information (Ω_t), which are not considered in the MARL-SR algorithm (Jang et al., 2023). The distribution of the latter is generated based on historical downtimes that depend on the machine type and the specific operations performed by each machine. A decision can only be made if a machine is available, meaning it is not processing, undergoing a setup change, or undergoing scheduled or unscheduled maintenance; otherwise, no action is taken. Subsequently, the followers make conversion decisions or maintain the current product type at every decision point, including the initial time of each shift, based on (15) and a rule-based conversion algorithm (detailed in Section 4.4). When a machine becomes idle, a job is routed – by default – to a machine already set up for the same product type, unless the agent actively initiates a changeover. Once all decisions by available followers are made, the statuses of the machines and WIP information are updated. At the end of each shift, rewards are assigned to each follower and the leader as specified in (16) and (19) respectively; however, no rewards are distributed during a shift. This is because the designated due time for meeting demand coincides with the end of the shift, resulting in a reward value of zero during the shift. As a result, after each shift, multiple (s, a, r) triplets corresponding to the number of decisions made by each follower are sampled, along with a single triplet for the leader. Each episode is stochastic with respect to the policy, demand, and unscheduled maintenance.

Although our goal is to enhance coordination by means of the leader, each agent functions individually, communicating via the goal vector. Consequently, samples are collected separately for each agent, as detailed in Algorithm 1. Therefore, we train each agent using their own rollouts. Specifically, we employ PPO algorithm (Schulman et al., 2017b) as the baseline algorithm to train all agents. PPO has received huge attention due to its simplicity, robustness for a large variety of tasks, low computational complexity, and good performance (Bohn et al., 2019; Zhang et al., 2019).

4.4. Rule-based conversion algorithm

Although a follower only needs to consider a single operation, there are usually numerous product candidates to choose from during a conversion. However, executing a conversion to start a product type can result in a significant loss of capacity, as it may take away the chance of producing other products due to the limited conversion time. To mitigate the risk of such a loss, we propose a rule-based conversion algorithm.

To determine the appropriate product type based on the rule-based conversion algorithm, we first score the urgency of product types for each operation using Algorithm 2. We begin by calculating the required capacity and expected remaining capacity for each product type. For a given product type p and operation o , the required capacity ($RC_{o,p}$)

Algorithm 1 A summary of simulation environment

Require: Episode duration NS , all producible product types p , sequences of operations for each product type $O_{p,j}, \forall p, j \in 1, 2, \dots, J_p$, sets of compatible machines $M_{p,j}, \forall p, j \in 1, 2, \dots, J_p$, unit processing times $PR_{p,j}, \forall p, j \in 1, 2, \dots, J_p$, scheduled maintenance information $SM_{l,t} \forall l, t$, and the conversion time threshold TH

- 1: **loop**
- 2: Configure the initial setup of each episode using job information ($D_{p,k}$ and $U_{p,k} \forall p, k \in 1, 2, \dots, K_p$), as well as the initial settings for machines ($Q_{l,0}, PT_{l,0}$, and $OP_{l,0} \forall l \in M$)
- 3: Sample random demand
- 4: The initial state vectors of the leader and the followers are set
- 5: **for** each shift **do**
- 6: **for** each decision point **do**
- 7: Sample unscheduled maintenance
- 8: **if** initial decision point **then**
- 9: The leader creates and distributes goal vectors based on (18)
- 10: **end if**
- 11: Machine statuses are updated to account for both scheduled and unscheduled maintenance based on (4)
- 12: All available followers take actions based on (15) and the rule-based conversion algorithm in Section 4.4
- 13: Machine statuses are updated to account for the actions of the followers based on (4)
- 14: **if** the end of the shift **then**
- 15: Rewards are assigned to each follower and the leader as specified in (16) and (19), respectively
- 16: The state vector of the leader is updated
- 17: **else**
- 18: Each follower receives a reward value of zero.
- 19: **end if**
- 20: The state vectors of the followers are updated
- 21: **end for**
- 22: **end for**
- 23: **end loop**

is the estimated total processing time for all target product WIPs in the target operation. The expected remaining capacity ($ERC_{o,p}$) is the estimated total processable time until the planning horizon ends of all machines that are currently processing p in operation o . If the required capacity is greater than the expected remaining capacity, we consider that product to be urgent and set the urgency score to the required capacity. Furthermore, a product that is not currently being processed by any machine in the operation is considered much more urgent because it cannot be produced without a conversion. In this case, we add a large value (BN) to the urgency score to prioritize this product. It is important to note that BN must be greater than the maximum value of the possible required capacity. The urgency score of o and p is denoted by $US_{o,p}$.

After urgency scoring computation in Algorithm 2, we make a conversion decision for each available machine as outlined in Algorithm 3. We first create an action for each operation o according to policy (15). Subsequently, we decode the corresponding action for each available machine to determine two key variables: the conversion indicator (CI), which indicates whether a conversion should be triggered, and the candidate product (p_{cand}), which represents the potential next product setup. When determining the next product for an available machine m , we take into account not only the generated action for that machine but also the current product WIPs and the operation's capacity for that current product. If there are WIPs for the selected product type, we assign p_{cand} to the next product setup in the episode. Otherwise, we prioritize urgent products and select the most urgent product as the

Algorithm 2 Urgency scoring

Require: Required capacity $RC_{o,p}$ and expected remaining capacity $ERC_{o,p}$ for all pairs of (operation o , product p)

- 1: Set a large value BN for urgency scoring
- 2: Initialize urgency score $US_{o,p} = 0$ for all pairs of (operation o , product p)
- 3: **for** each operation o **do**
- 4: **for** each product type p processable in operation o **do**
- 5: **if** $RC_{o,p} > ERC_{o,p}$ **then**
- 6: **if** p is not being processed by any machine in operation o **then**
- 7: $US_{o,p} = RC_{o,p} + BN$
- 8: **else**
- 9: $US_{o,p} = RC_{o,p}$
- 10: **end if**
- 11: **end if**
- 12: **end for**
- 13: **end for**

next product setup. To accomplish this, we evaluate the current product setup p_m on machine m and determine if it can be replaced with the most urgent product, $\text{argmax}_{p \in P_o} US_{o,p}$, where P_o is the set of eligible products by operation o . To ensure that there will be no missing lots of p_m , we first verify that all WIPs of p_m can be completed before the planning horizon ends. We do this by verifying that the required capacity RC_{o,p_m} is less than $ERC_{o,p_m} - ERCM_{o,m,p_m}$, the expected remaining capacity without the target machine m . Here, $ERCM_{o,m,p_m}$ refers to the expected remaining capacity of the target machine m . Additionally, we check if there is an urgent product ($\sum_{p \in P_o} US_{o,p} > 0$). If both conditions are met, we assign the most urgent product as the next product for that machine.

Illustrative examples: Suppose operation 5 has two machines both set to process product A. A follower agent attempts to convert one machine to product B due to temporary demand. However, this conversion uses up nearly all of the shift's changeover time budget. As a result, the machine cannot be reverted back to product A within the same shift. Since product A still has high WIP, this decision would cause a significant production loss. The rule-based algorithm detects this risk – by checking the required capacity for product A and verifying that the remaining capacity is insufficient if the conversion proceeds – and blocks the follower's decision. This safeguard prevents irreversible decisions that could lead to large-scale idle time and unmet demand.

Conversely, suppose no machines in operation 6 are currently set up for product C, which has an urgent lot with an imminent due time. The follower does not suggest conversion, possibly due to low immediate reward or limited WIP. However, the rule-based algorithm detects that the required capacity for product C significantly exceeds the remaining capacity of other machines and that missing this conversion would cause severe delay. By assigning a high urgency score to product C, the algorithm overrides the follower's inaction and forces a conversion, ensuring timely processing and preventing deadline violations.

In summary, we introduce several key modifications to the MARL-SR algorithm outlined in Jang et al. (2023):

- MARL-SR employs synthetic rewards to distinguish and reflect each agent's contribution to the overall team reward, as individual agent reward calculation is typically infeasible. In contrast, for factory scheduling, we can measure operation-wise performance, allowing us to implement an operation-specific reward calculation.
- Additionally, our approach models both scheduled and unscheduled machine maintenance, which are not considered in MARL-SR.

Algorithm 3 Conversion decision at t in shift n

Require: Required capacity $RC_{o,p}$ and expected remaining capacity of operation $ERC_{o,p}$ for all pairs of (operation o , product p), expected remaining capacity of machine $ERCM_{o,m,p}$ for all triplets of (operation o , machine m , product p)

- 1: *Urgency scoring()*
- 2: **for** each operation o **do**
- 3: **if** there is an available machine **then**
- 4: Produce $a_{n,t}^o = \{a_{n,t}^{o,m} | m \in \text{machines for operation } o\}$
- 5: **for** each available machine m **do**
- 6: Get current product setup p_m
- 7: CI and $p_{cand} \leftarrow \text{Decode}(a_{n,t}^{o,m})$
- 8: **if** $CI = \text{True}$ **then**
- 9: **if** there are WIPs of p_{cand} in operation o **then**
- 10: Next product setup $p_m \leftarrow p_{cand}$
- 11: **else if** $RC_{o,p_m} < ERC_{o,p_m} - ERCM_{o,m,p_m}$ and $\sum_{p \in P_o} US_{o,p} > 0$ **then**
- 12: Next product setup $p_m \leftarrow \text{argmax}_{p \in P_o} US_{o,p}$
- 13: **else**
- 14: Do not change product setup
- 15: **end if**
- 16: **end if**
- 17: **end for**
- 18: **end if**
- 19: **end for**

- Given that poor decisions by an agent can severely impact production capacity in factory scheduling, we also develop a rule-based conversion algorithm to override agent decisions if they could lead to significant production losses.

5. Experimental evaluations**5.1. Implementation details**

We implemented both the proposed model and the benchmark models using the PPO algorithm. For each agent, a fully-connected neural network, consisting of two layers with 256 units each and ReLU activation functions, is utilized for both the actor and the critic components. In our algorithm, the actor consists of the leader and the followers which all have such network architecture. We compared our proposed model with four benchmark models, which are described in detail later in this section. To determine the optimal hyperparameters for each model, we conducted both Kruskal–Wallis and ANOVA tests for reasonable parameter choices and groups corresponding to performance metrics. The results showed no significant differences across the four performance metrics—tardiness, number of changeovers, cumulative idle time, and completion rate against demand—within the parameter ranges tested: batch size (64–256), learning rate (0.0001–0.00001), discount factor (0.9–0.99), and lambda (0.9–0.99) for our proposed model and three of the benchmark models. For the other benchmark model, performance improvement was observed only when lambda was set to 0.95, with no statistical differences noted for the other hyperparameters. Consequently, we applied the hyperparameter settings listed in Table 1 to all models. Additionally, for each operation o in shift n , the dimension of the corresponding goal vector g_n^o is set to 3, as recommended in Jang et al. (2023). All models compared in this study are implemented using TensorFlow version 2.8 on a server equipped with an Intel i9-12900k CPU. Due to the shallow network architecture employed for both the actor and the critics, and the simulation's need for extensive sequential computations, GPU execution does not help.

In this study, we evaluate the effectiveness of our proposed method in real-world scheduling scenarios using two simulators built on different real production datasets for Intel's high volume packaging and

Table 1
Hyperparameters.

Hyperparameter	Value
Batch size	256
Learning rate	0.0001
Discount factor	0.9900
Clipping value	0.2000
Generalized advantage estimation (GAE) parameter lambda	0.9500

Table 2
Summary of two scenarios.

Scenario	Short-term	Long-term
No. product types	35	20
No. operations	26	21
No. machines	159	115
Avg. out-degree in DAG	1.33	1.37
Duration	1 week (14 shifts)	3 weeks (42 shifts)

test factory. We refer to the two production scenarios as short-term and long-term scenarios, as summarized in Table 2. Here, the average out-degree in DAG refers to the mean number of outgoing arcs in the DAG, which is constructed based on the precedence constraints of operations. To assess performance in diverse environments, we further diversified each scenario into low-demand, medium-demand, and high-demand cases. Specifically, to evaluate model robustness, we apply the model trained under the low-demand case to all three demand levels, where the medium- and high-demand cases impose approximately threefold and fivefold increases in demand, respectively.

To enhance exploration during training, the set of target product types required for production in each episode changes probabilistically, resulting in varying workloads for each agent. Additionally, to promote exploration within the PPO algorithm, we incorporate entropy regularization as outlined in Schulman et al. (2017a), with the entropy coefficient of 0.01. This regularization helps maintain policy diversity by preventing premature convergence and fostering a broader exploration of the action space.

For each demand case in both short- and long-term scenarios, the comparison models are trained for 10,000 and 50,000 episodes, respectively. Each model adopts the policy that achieved the highest team reward within the last 100 episodes of training. These optimal models are then tested over 100 episodes. In each test episode, all comparison models begin scheduling under identical conditions, including the same demand information as well as both scheduled and unscheduled maintenance details.

In our algorithm, the followers learn policies to meet the demand over a predefined number of shifts. However, the number of shifts in the two cases in Table 2 is too long to enable effective learning within a reasonable timeframe. Therefore, we train the agents in simulations spanning four shifts (two days) and then apply the learned policies using a rolling horizon approach during inference. In the long-term scenario, training takes 1.68 h, with inference requiring 40.14 s per episode. For the short-term scenario, training takes 49.02 min, and inference takes 14.08 s per episode. Given the planning horizon for both scenarios, the time required for both training and inference is manageable.

In this study, we compare the proposed model with existing RL-based dynamic scheduling models. While several such models have been developed, most are not suitable for factory-wide scheduling because existing models do not consider factory-specific operational constraints in modeling or require excessive computational time.

Four recent models – CCC (Liu et al., 2020), JDYWH (Chang et al., 2022), RRC (Liu et al., 2023), and MYFM (Xu et al., 2024) – can be adapted for factory-wide scheduling with some modifications. For clarity, we refer to each model using the initials of all authors' first names,

as their original model names are quite similar. The four comparison models use RL agents to select dispatching rules at each action point. CCC assigns one agent per operation to select from three dispatching rules. JDYWH uses a single agent to handle all operations and chooses among four dispatching rules. RRC also assigns one agent per operation but selects from four dispatching rules. MYFM introduces two agents for all operations: one to select a rule from four routing options and another to select a rule from four dispatching options. However, as the third rule corresponding to JDYWH is not applicable in our problem, we excluded it from the analysis.

In a factory environment, the large state and action vectors necessary to manage hundreds of machines lead to significant learning and inference times. Moreover, the state representation and reward calculation methods used in the four comparison models are not directly applicable to our simulation, where the number of jobs demanded varies and the penalties for tardy jobs are unknown. Therefore, we employ the same multi-agent setting where an agent is assigned to each operation (with the concept of a leader being unique to our model), use a consistent state representation feature format (with our model incorporating goal vectors from the leader), and apply an operation-wise rewarding strategy across all models, focusing on comparing the algorithmic differences in job selection and agent coordination.

5.2. Comparison with benchmarks

In this section, we compare the proposed model with existing RL-based dynamic scheduling models. Fig. 3 presents the learning curves of the proposed model and four baseline models under long-term and short-term production scenarios. In the figures, the moving window method is applied, where the team rewards are averaged over a sliding window of 1000 episodes. The averaging window is shifted by one episode at a time, indicating a step size of one episode per move. In all scenarios – except when the number of delayed lots is used as the reward in the short-term production setting (Fig. 3(c)) – all models exhibit improved policies over time, as indicated by rising total rewards. However, the four baseline models converge prematurely and plateau at lower reward levels, whereas the proposed model continues to improve and ultimately achieves the highest cumulative reward in both scenarios. Notably, in Fig. 3(c), the baseline models fail to make meaningful progress, whereas the proposed model still shows significant performance gains, demonstrating its robustness even under challenging reward formulations.

Next, we utilize four metrics, namely tardiness (Ta), the number of changeovers (NC), cumulative idle time (CIT), and completion rate (CR) against demand, to compare the models. We are unable to divulge the performance of the proposed model due to Intel's data confidentiality policy. Consequently, we establish the proposed model as the baseline and assess the improvement (percentage improvement) over benchmark models. Specifically, we report an increase in completion rate and a decrease in tardiness, the number of changeovers, and cumulative idle time as performance improvement. In summary, a negative number means that we outperform.

Figs. 4(a) and 4(b) present the percentage improvement of the four comparison models over our proposed model in the long-term production scenario, using the number of delayed lots and tardiness as reward signals, respectively. For each metric, we highlight the improvement values in bold when the proposed model achieves the best performance. Overall, the proposed model outperforms all baselines in 11 out of 12 criteria in Fig. 4(a) and 9 out of 12 in Fig. 4(b). Notably, as demand increases from low to medium and high, the superiority and robustness of our model become more pronounced—achieving the best results across all criteria in the high-demand setting. Although the proposed model shows a relative underperformance in CIT under the low-demand case in Fig. 4(a), it excels in more critical metrics such as tardiness and completion rate, which are more directly related to productivity. Similarly, the slightly reduced performance in tardiness

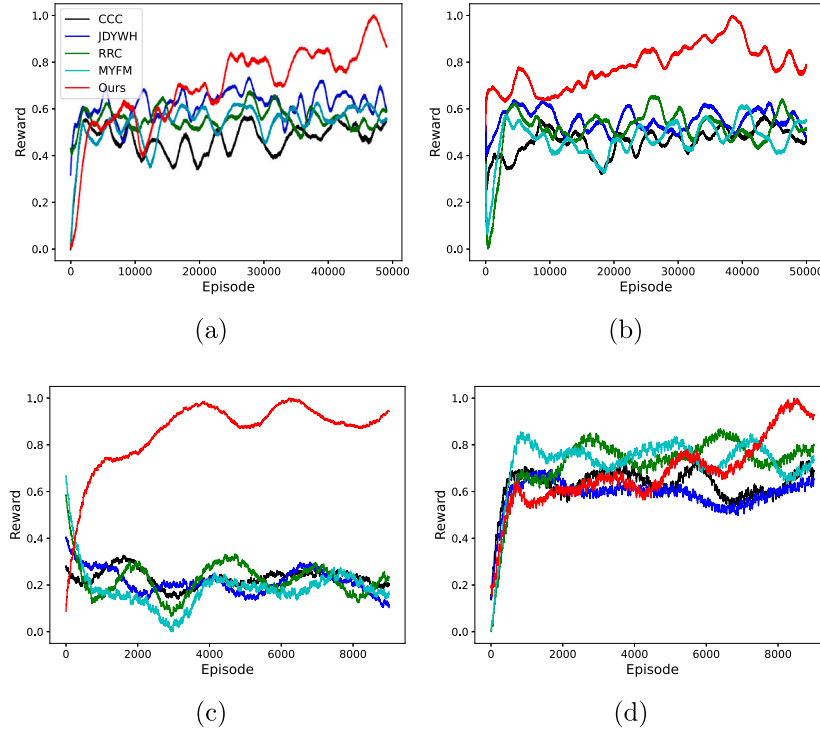


Fig. 3. Learning curve of the five models: (a) using the number of delayed lots (b) using tardiness as the reward signal under the long-term production scenario (c) using the number of delayed lots (d) using tardiness as the reward signal under the short-term production scenario. Min–max normalization is applied to the total reward to standardize the scale of the y-axis.

under low demand in Fig. 4(b) is negligible, as all models exhibit very low tardiness in this setting—evidenced by the high standard deviations (86.69%–152.78%), which indicate that differences in absolute performance are marginal.

Similarly, Figs. 4(c) and 4(d) summarize the percentage improvement of the four comparison models over our proposed model in the short-term production scenario, using the number of delayed lots and tardiness as reward signals, respectively. The proposed model clearly outperforms the baselines in all criteria in Fig. 4(c), and 11 out of 12 in Fig. 4(d), demonstrating strong generalization even in the short-horizon setting. Although the proposed model performs slightly worse in CR under the low-demand case in Fig. 4(d), where CCC and MYFM exhibit marginally better results, it consistently outperforms these models – as well as the other baselines – under medium and high demand conditions. This highlights the robustness of the proposed model, particularly in more challenging scenarios. Notably, in all four figures in Fig. 4, the relative improvement of our model in terms of CR consistently increases with higher demand levels, further demonstrating its scalability and adaptability to varying production loads.

Although our proposed model outperforms the comparison models, it is essential to demonstrate its generalizability to other benchmark datasets, as RL models can be highly sensitive to dataset variations. To address this, we develop a new virtual scheduling environment based on the setup from Liu et al. (2023). This environment includes four consecutive operations, with 3, 2, 3, and 2 machines assigned to each operation, respectively. There are 15 product types, each requiring two to three operations. The processing time for each operation is randomly sampled from a uniform distribution between 1 and 10 min. To assess the model’s robustness to changeover, we create two versions of the environment with different changeover time distributions: an easy case with a uniform distribution between 10 and 20 min, and a more difficult case with a uniform distribution between 1 and 2 h. The planning horizon is set to 12 h. Maintenance is not included, as the comparison models do not account for it.

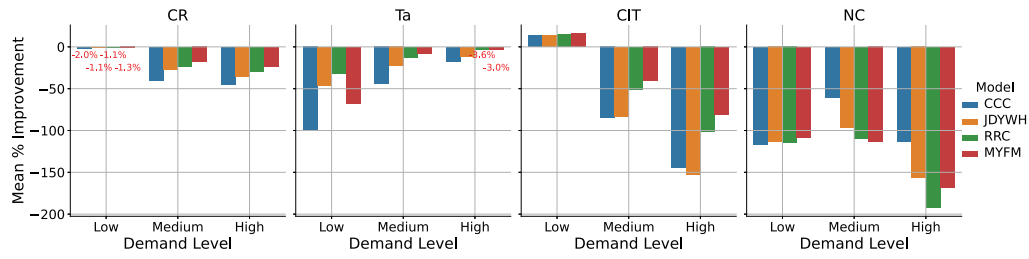
In this virtual environment, we use the number of delayed lots as the reward signal. In the easy case, the corresponding learning curves are shown in Fig. 5(a). All baseline models except MYFM exhibit generally increasing reward trends. Our model shows a small performance gap compared to CCC and JDYWH in terms of maximum cumulative reward. As shown in Table 3, when evaluated on 100 unseen scheduling scenarios, our model achieves slightly lower CR compared to CCC and RRC, indicating comparable but not superior generalization under scenarios with short changeover durations.

In contrast, when the environment becomes more challenging due to longer changeover times (Fig. 5(b)), the consequences of poor changeover decisions become much more pronounced, often leading to severe production loss. In this difficult case, our proposed model demonstrates clear superiority across all metrics, with only a marginal increase in NC. These results highlight the robustness and adaptability of our model in complex and disturbance-prone production environments, where effective changeover planning is critical.

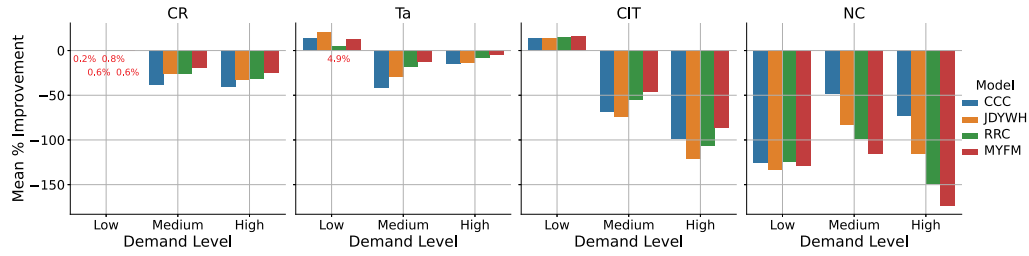
5.3. Ablation study

To verify the effectiveness of each component proposed in this study, we conduct a quantitative ablation study. Specifically, we generate 100 production plans based on the long-term scenario outlined in Table 2. We then compare the performance of the following models:

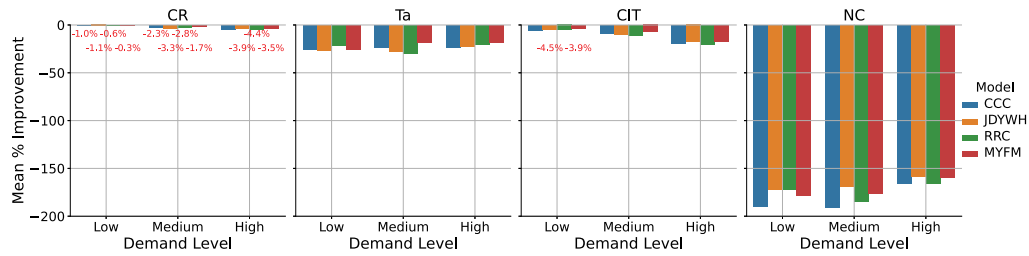
- (1) Shared reward model (SRM): Under this model, the followers are updated using a reward-sharing strategy, where the rewards are calculated based on all delayed lots and distributed equally among all followers.
- (2) Operation-wise reward model (ORM): This model calculates rewards for each follower in the assigned operation.
- (3) Leader–follower shared reward model (LFSRM): The leader–follower concept using the abstract goal is incorporated into SRM.



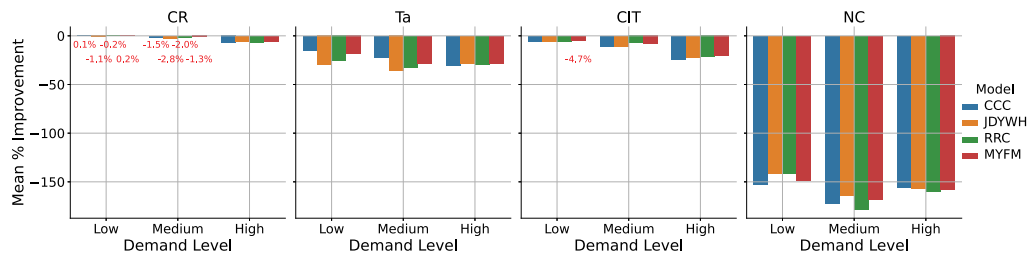
(a) Reward signal: the number of delayed lots in long-term production



(b) Reward signal: tardiness in long-term production



(c) Reward signal: the number of delayed lots in short-term production



(d) Reward signal: tardiness in short-term production

Fig. 4. Percentage improvement of comparison models in four metrics compared to the proposed model.

Table 3

Comparison in terms of the four metrics compared to the proposed model for the virtual scheduling environment, using the number of delayed lots as the reward signal.

Demand	Metric	Ours		CCC		JDYWH		RRC		MYFM	
		Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.
Easy	Ta (sec/lot)	29.62	40.35	24.55	31.14	40.50	40.85	13.24	15.68	29.03	34.49
	NC	8.41	2.91	14.01	3.24	14.23	2.86	14.72	3.06	14.59	2.90
	CIT (hrs)	7.70	1.18	7.75	1.21	7.96	1.13	7.81	1.05	8.03	1.09
	CR (%)	77.06	24.51	81.17	21.25	68.88	27.20	85.35	17.48	74.48	28.59
Difficult	Ta (sec/lot)	67.94	60.45	87.09	65.63	95.45	71.53	92.75	57.83	96.64	72.42
	NC	10.82	20.03	8.5	11.04	8.6	13.12	8.25	8.27	7.73	9.43
	CIT (hrs)	5.46	1.84	5.82	1.30	5.97	1.45	5.80	1.43	5.79	1.43
	CR (%)	58.09	31.55	46.74	30.25	45.30	29.78	41.71	26.65	44.08	27.85

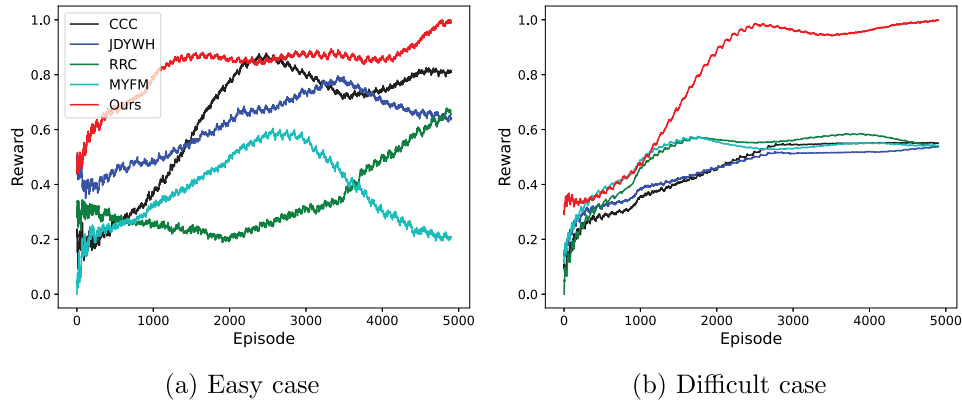


Fig. 5. Learning curves of the five models on the virtual scheduling environment using the number of delayed lots as the reward signal: (a) easy case and (b) difficult case. Min-max normalization is applied to the total reward to standardize the scale of the y-axis.

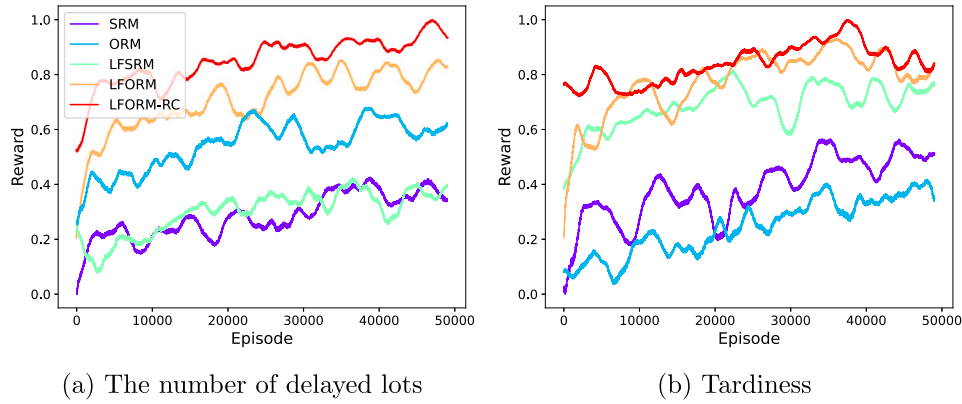


Fig. 6. Learning curve of the five baseline models used in the ablation study: (a) using the number of delayed lots, and (b) using tardiness as the reward signal. Min-max normalization is applied to the total reward to standardize the scale of the y-axis.

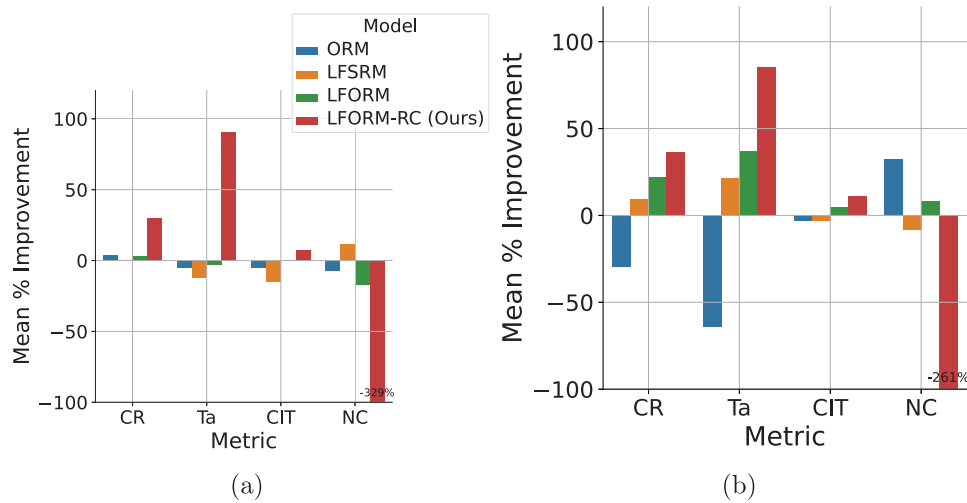


Fig. 7. Percentage improvement in four metrics compared to SRM after conducting 100 simulations and averaging the results (a) using the number of delayed lots as the reward signal and (b) using tardiness as the reward signal.

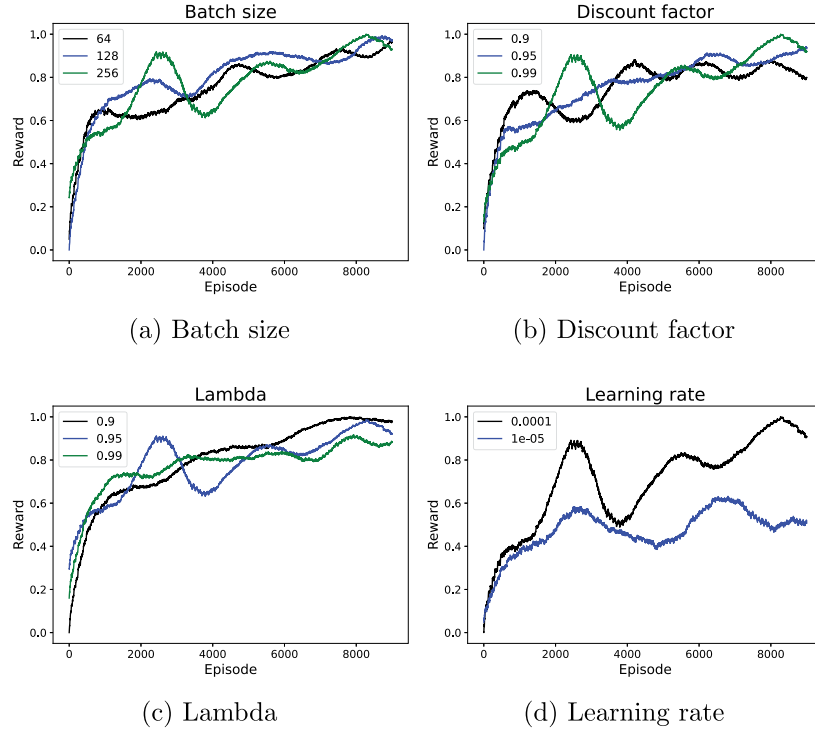


Fig. 8. Sensitivity analysis results with respect to variations in (a) batch size, (b) discount factor, (c) lambda, and (d) learning rate, using the number of delayed lots as the reward signal. For each hyperparameter, all other parameters are fixed to the default values listed in [Table 1](#).

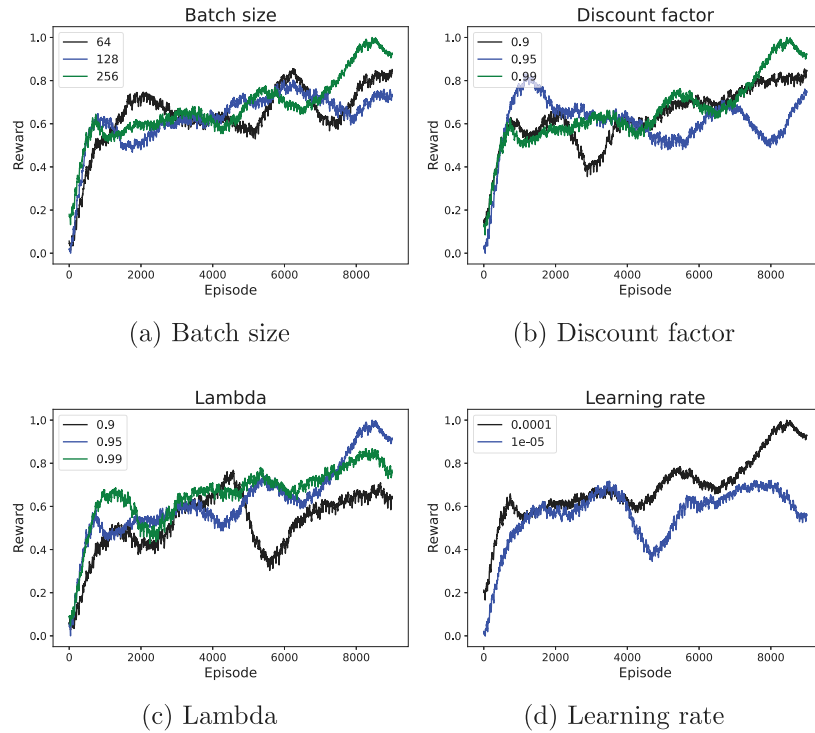


Fig. 9. Sensitivity analysis results with respect to variations in (a) batch size, (b) discount factor, (c) lambda, and (d) learning rate, using tardiness as the reward signal. For each hyperparameter, all other parameters are fixed to the default values listed in [Table 1](#).

(4) Leader–follower operation-wise reward model (LFORM): The leader–follower concept is incorporated into ORM. Specifically, LFORM introduces operation-wise reward calculation to the LFSRM framework.

(5) Proposed model: The proposed model, LFORM-RC, incorporates rule-based conversion in addition to the LFORM approach. The acronym RC signifies the inclusion of rule-based conversion as a novel feature of the model.

We begin by comparing the learning curves of the baseline models using the number of delayed lots and tardiness as reward signals, as shown in Figs. 6(a) and 6(b), respectively. Overall, both the operation-wise reward strategy and the proposed leader–follower MARL framework – where abstract goals are used for agent coordination – contribute to enhanced learning performance. Notably, the integration of the rule-based conversion (RC) mechanism further accelerates early learning by mitigating catastrophic drops in production capability at the initial stage. This stabilizing effect not only improves the starting point of training but also enables the model to achieve the highest cumulative reward over time.

We establish SRM as the baseline and assess the improvement of the other models based on the four metrics used in Section 5.2 compared to the baseline. The comparison results are presented in Fig. 7. It is important to note that, while a small number of conversions is desirable as it can reduce operating costs, other metrics such as tardiness and completion rate take priority, as they are crucial for achieving high productivity. The results demonstrate that the operation-wise reward strategy, the leader–follower concept using abstract goals, and the rule-based conversion contribute to improved performance on the target metric used as the reward signal. In particular, the rule-based conversion is crucial in improving tardiness and completion rates by preventing significant capacity losses through the minimization of poor conversions.

5.4. Sensitivity analysis

To evaluate the robustness of the proposed model with respect to key hyperparameters, we conducted a sensitivity analysis under the short-term production scenario using two reward signals: the number of delayed lots and tardiness. The hyperparameters considered in this study include: (a) batch size, (b) discount factor, (c) GAE parameter λ , and (d) learning rate. For each case, only one hyperparameter is varied while keeping all others fixed to the default values listed in Table 1. The results presented in Figs. 8 and 9 demonstrate that the proposed model exhibits stable and steadily improving performance across most configurations, indicating strong robustness to hyperparameter variations. Among the hyperparameters, the learning rate has the most pronounced impact: an excessively small value (e.g., 10^{-5} , labeled as $1e-05$ in the figure) leads to significantly slower convergence and lower final performance. Although the discount factor and λ also influence training dynamics, their effects are relatively minor in terms of final reward. Nevertheless, careful tuning of these parameters – particularly the learning rate – is beneficial for accelerating convergence and improving training efficiency.

6. Conclusion

In this study, we present a novel scalable MARL model for dynamic scheduling problems in real-world factories. The proposed model addresses several hurdles faced in factory scheduling by combining operation-specific agent modeling, abstract goal-based leader–follower coordination, and a rule-based conversion algorithm, which are novel in the field of RL-based scheduling. In particular, we found that RL agents can make errors that lead to significant losses in production capacity. To mitigate this issue, a rule-based conversion algorithm can be used to supplement the agents' operational decisions. Our experiments, conducted on simulators constructed using Intel production datasets, demonstrate that the proposed model significantly outperforms state-of-the-art RL-based scheduling models that rely on human-made dispatching rules. This highlights the limitations of existing methods that use a combination of human-made rules and the potential of our approach to provide more efficient scheduling solutions for real-world factories.

While our proposed model exhibits good performance and scalability, there is an issue that needs to be addressed in our future research.

The model assumes that production-related information, such as operation list, machine list, and product types, remain fixed. However, in real-world factories, these settings can change from time to time, necessitating a retraining of the model. Although this can be easily addressed with sufficient computing resources, the process requires frequent manual interventions, which can be a challenge. To overcome this, we need to develop a dynamically evolving model that can adapt to factory setting changes. This direction will be a focus of our future research efforts.

CRedit authorship contribution statement

Jaeyeon Jang: Writing – original draft, Visualization, Validation, Methodology, Conceptualization. **Diego Klabjan:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Han Liu:** Writing – review & editing. **Nital S. Patel:** Project administration, Data curation. **Xiuqi Li:** Software, Investigation. **Balakrishnan Ananthanarayanan:** Software, Data curation. **Husam Dauod:** Software, Investigation. **Tzung-Han Juang:** Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by a Semiconductor Research Corporation program sponsored by Intel, USA.

Data availability

The data that has been used is confidential.

References

- Amjad, M.K., Butt, S.I., Kousar, R., Ahmad, R., Agha, M.H., Faping, Z., Anjum, N., Asgher, U., 2018. Recent research trends in genetic algorithm based flexible job shop scheduling problems. *Math. Probl. Eng.* 2018, 1–32. <http://dx.doi.org/10.1155/2018/9270802>.
- Aydin, M., Öztemel, E., 2000. Dynamic job-shop scheduling using reinforcement learning agents. *Robot. Auton. Syst.* 33 (2–3), 169–178. [http://dx.doi.org/10.1016/S0921-8890\(00\)00087-7](http://dx.doi.org/10.1016/S0921-8890(00)00087-7).
- Bøhn, E., Coates, E.M., Moe, S., Johansen, T.A., 2019. Deep reinforcement learning attitude control of fixed-wing UAVs using proximal policy optimization. In: *Proceedings of International Conference on Unmanned Systems*. pp. 523–533. <http://dx.doi.org/10.1109/ICUS50048.2020.9274875>.
- Branke, J., Hildebrandt, T., Scholz-Reiter, B., 2015. Hyper-heuristic Evolution of dispatching rules: A comparison of rule representations. *Evol. Comput.* 23 (2), 249–277. http://dx.doi.org/10.1162/EVCO_a_00131.
- Chang, J., Yu, D., Hu, Y., He, W., Yu, H., 2022. Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival. *Processes* 10, 760. <http://dx.doi.org/10.3390/pr10040760>.
- Chen, Y., Su, L.H., Tsai, Y.C., Huang, S., Chou, F.D., 2019. Scheduling jobs on a single machine with dirt cleaning consideration to minimize total completion time. *IEEE Access* 7, 22290–22300. <http://dx.doi.org/10.1109/ACCESS.2019.2898905>.
- Cui, Y., Matsubara, T., Sugimoto, K., 2017. Pneumatic artificial muscle-driven robot control using local update reinforcement learning. *Adv. Robot.* 31 (8), 397–412. <http://dx.doi.org/10.1080/01691864.2016.1274680>.
- Cunha, B., Madureira, A.M., Fonseca, B., Coelho, D., Madureira, A.M., Abraham, A., Gandhi, N., Varela, M.L., 2020. Deep reinforcement learning as a job shop scheduling solver: A literature review. In: *Hybrid Intelligent Systems*, vol. 923, Springer.
- Dehghan-Saneji, K., Eghbali-Zarch, M., Tavakkoli-Moghaddam, R., Sajadi, S.M., Sadjaji, S.J., 2021. Solving a new robust reverse job shop scheduling problem by meta-heuristic algorithms. *Eng. Appl. Artif. Intell.* 101, 104207. <http://dx.doi.org/10.1016/j.engappai.2021.104207>, URL <https://www.sciencedirect.com/science/article/pii/S0952197621000543>.
- Fattahi, P., Saidi Mehrabad, M., Jolai, F., 2007. Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *J. Intell. Manuf.* 18 (3), 331–342. <http://dx.doi.org/10.1007/s10845-007-0026-8>.

- Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., Whiteson, S., 2018. Counterfactual multi-agent policy gradients. In: AAAI Conference on Artificial Intelligence.
- Gao, K., Cao, Z., Zhang, L., Chen, Z., Han, Y., Pan, Q., 2019a. A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems. *IEEE/CAA J. Autom. Sin.* 6 (4), 904–916. <http://dx.doi.org/10.1109/JAS.2019.1911540>.
- Gao, K.Z., Suganthan, P.N., Chua, T.J., Chong, C.S., Cai, T.X., Pan, Q.K., 2015. A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion. *Expert Syst. Appl.* 42 (21), 7652–7663. <http://dx.doi.org/10.1016/j.eswa.2015.06.004>.
- Gao, K., Yang, F., Zhou, M., Pan, Q., Suganthan, P.N., 2019b. Flexible job-shop rescheduling for new job insertion by using discrete jaya algorithm. *IEEE Trans. Cybern.* 49, 1944–1955. <http://dx.doi.org/10.1109/TCYB.2018.2817240>.
- Gui, Y., Tang, D., Zhu, H., Zhang, Y., Zhang, Z., 2023. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Comput. Ind. Eng.* 180, 109255.
- Gui, Y., Zhang, Z., Tang, D., Zhu, H., Zhang, Y., 2024. Collaborative dynamic scheduling in a self-organizing manufacturing system using multi-agent reinforcement learning. *Adv. Eng. Inform.* 62, 102646.
- Guo, X., Liu, S., Zhou, M., Tian, G., 2016. Disassembly sequence optimization for large-scale products with multiresource constraints using scatter search and Petri nets. *IEEE Trans. Cybern.* 46, 2435–2446. <http://dx.doi.org/10.1109/TCYB.2015.2478486>.
- Hein, D., Depeweg, S., Tokic, M., Udluft, S., Hentschel, A., Runkler, T.A., Sterzing, V., 2018. A benchmark environment motivated by industrial control problems. In: *IEEE Symposium Series on Computational Intelligence*. pp. 1–8. <http://dx.doi.org/10.1109/SSCI.2017.8280935>, [arXiv:1709.09480](https://arxiv.org/abs/1709.09480).
- Hu, L., Liu, Z., Hu, W., Wang, Y., Tan, J., Wu, F., 2020. Petri-net-based dynamic scheduling of flexible manufacturing system via deep reinforcement learning with graph convolutional network. *J. Manuf. Syst.* 55, 1–14. <http://dx.doi.org/10.1016/j.jmsy.2020.02.004>.
- Jang, J., Klabjan, D., Liu, H., Patel, N.S., Li, X., Ananthanarayanan, B., Dauod, H., Juang, T.H., 2023. Learning multiple coordinated agents under directed acyclic graph constraints. *arXiv preprint arXiv:2307.07529*.
- Jiang, J., Lu, Z., 2021. The emergence of individuality. In: Meila, M., Zhang, T. (Eds.), *ICML*, In: *International Conference on Machine Learning*, vol. 139, pp. 4992–5001.
- Kacem, I., Hammadi, S., Borne, P., 2002. Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Trans. Syst. Man Cybern. Part C (Applications Reviews)* 32, 1–13. <http://dx.doi.org/10.1109/TSMCC.2002.1009117>.
- Kim, D.K., Kim, H.J., Lee, T.E., 2017. Optimal scheduling for sequentially connected cluster tools with dual-armed robots and a single input and output module. *Int. J. Prod. Res.* 55 (11), 3092–3109. <http://dx.doi.org/10.1080/00207543.2016.1243819>.
- Kraemer, L., Banerjee, B., 2016. Multi-agent reinforcement learning as a rehearsal for decentralized planning. *Neurocomputing* 190, 82–94. <http://dx.doi.org/10.1016/j.neucom.2016.01.031>.
- Kundakci, N., Kulak, O., 2016. Hybrid genetic algorithms for minimizing makespan in dynamic job shop scheduling problem. *Comput. Ind. Eng.* 96, 31–51. <http://dx.doi.org/10.1016/j.cie.2016.03.011>.
- Li, X., Gao, L., 2020. A hybrid genetic algorithm and tabu search for multi-objective dynamic JSP. In: *Effective Methods for Integrated Process Planning and Scheduling*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 377–403. http://dx.doi.org/10.1007/978-3-662-55305-3_18.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2016. Continuous control with deep reinforcement learning. In: *International Conference on Learning Representations*. pp. 1–14.
- Liu, C.L., Chang, C.C., Tseng, C.J., 2020. Actor-critic deep reinforcement learning for solving job shop scheduling problems. *IEEE Access* 8, 71752–71762. <http://dx.doi.org/10.1109/ACCESS.2020.2987820>.
- Liu, R., Piplani, R., Toro, C., 2023. A deep multi-agent reinforcement learning approach to solve dynamic job shop scheduling problem. *Comput. Oper. Res.* 159, 106294.
- Lou, P., Liu, Q., Zhou, Z., Wang, H., Sun, S.X., 2012. Multi-agent-based proactive-reactive scheduling for a job shop. *Int. J. Adv. Manuf. Technol.* 59, 311–324. <http://dx.doi.org/10.1007/s00170-011-3482-4>.
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., Mordatch, I., 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. In: *Advances in Neural Information Processing Systems*. pp. 6382–6393, [arXiv:1706.02275](https://arxiv.org/abs/1706.02275).
- Luo, S., 2020. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Appl. Soft Comput.* 91, 106208. <http://dx.doi.org/10.1016/j.asoc.2020.106208>.
- Luo, S., Zhang, L., Fan, Y., 2021. Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Comput. Ind. Eng.* 159, 107489. <http://dx.doi.org/10.1016/j.cie.2021.107489>.
- Min, B., Kim, C.O., 2022. State-dependent parameter tuning of the apparent tardiness cost dispatching rule using deep reinforcement learning. *IEEE Access* 10, 20187–20198. <http://dx.doi.org/10.1109/ACCESS.2022.3152192>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human-level control through deep reinforcement learning. *Nature* 518 (7540), 529–533.
- Nguyen, S., Zhang, M., Johnston, M., Tan, K.C., 2013. A computational study of representations in genetic programming to evolve dispatching rules for the job shop scheduling problem. *IEEE Trans. Evol. Comput.* 17 (5), 621–639. <http://dx.doi.org/10.1109/TEVC.2012.2227326>.
- Ning, T., Huang, M., Liang, X., Jin, H., 2016. A novel dynamic scheduling strategy for solving flexible job-shop problems. *J. Ambient. Intell. Humaniz. Comput.* 7, 721–729. <http://dx.doi.org/10.1007/s12652-016-0370-7>.
- Oliehoek, F.A., Vlassis, N., 2007. Q-value functions for decentralized POMDPs. In: *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems*. pp. 838–845.
- Ouelhadj, D., Petrovic, S., 2009. A survey of dynamic scheduling in manufacturing systems. *J. Sched.* 12, 417–431. <http://dx.doi.org/10.1007/s10951-008-0090-8>.
- Park, J., Chun, J., Kim, S.H., Kim, Y., and, J.P., 2021. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *Int. J. Prod. Res.* 59 (11), 3360–3377. <http://dx.doi.org/10.1080/00207543.2020.1870013>.
- Rashid, T., Samvelyan, M., Schroeder, C., Farquhar, G., Foerster, J., Whiteson, S., 2018. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In: *International Conference on Machine Learning*. pp. 4295–4304.
- Schulman, J., Abbeel, P., Chen, X., 2017a. Equivalence between policy gradients and soft Q-learning. [arXiv:1704.06440](https://arxiv.org/abs/1704.06440).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017b. Proximal policy optimization algorithms. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- Shady, S., Kaihara, T., Fujii, N., Kokuryo, D., 2022. A novel feature selection for evolving compact dispatching rules using genetic programming for dynamic job shop scheduling. *Int. J. Prod. Res.* 60 (13), 4025–4048. <http://dx.doi.org/10.1080/00207543.2022.2053603>.
- Shahrabi, J., Adibi, M.A., Mahootchi, M., 2017. A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. *Comput. Ind. Eng.* 110, 75–82. <http://dx.doi.org/10.1016/j.cie.2017.05.026>.
- Shiue, Y.R., Lee, K.C., Su, C.T., 2021. Development of dynamic scheduling in semiconductor manufacturing using a Q-learning approach. *Int. J. Comput. Integr. Manuf.* 1–17. <http://dx.doi.org/10.1080/0951192X.2021.1946849>.
- Shu, T., Tian, Y., 2019. M³RL: Mind-aware multi-agent management reinforcement learning. In: *International Conference on Learning Representations*. [arXiv:1810.00147](https://arxiv.org/abs/1810.00147).
- Subramanian, K., Isbell, Jr., C.L., Thomaz, A.L., 2016. Exploration from demonstration for interactive reinforcement learning. In: *International Conference on Autonomous Agents and Multiagent Systems*.
- Sutton, R.S., Barto, A.G., 2018. *Reinforcement Learning*. MIT Press.
- Tan, M., 1993. Multi-agent reinforcement learning: Independent vs. cooperative agents. In: *International Conference on Machine Learning*. pp. 330–337. <http://dx.doi.org/10.1016/b978-1-55860-307-3.50049-6>.
- Vepsäläinen, A.P.J., Morton, T.E., 1987. Priority rules for job shops with weighted tardiness costs. *Manag. Sci.* 33, 1035–1047. <http://dx.doi.org/10.1287/mnsc.33.8.1035>.
- Wang, L., Hu, X., Wang, Y., Xu, S., Ma, S., Yang, K., Liu, Z., Wang, W., 2021. Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning. *Comput. Netw.* 190, 107969. <http://dx.doi.org/10.1016/j.comnet.2021.107969>, URL <https://www.sciencedirect.com/science/article/pii/S1389128621001031>.
- Wang, Y.C., Usher, J.M., 2005. Application of reinforcement learning for agent-based production scheduling. *Eng. Appl. Artif. Intell.* 18 (1), 73–82. <http://dx.doi.org/10.1016/j.engappai.2004.08.018>.
- Wang, Z., Zhang, J., Yang, S., 2019. An improved particle swarm optimization algorithm for dynamic job shop scheduling problems with random job arrivals. *Swarm Evol. Comput.* 51, 100594. <http://dx.doi.org/10.1016/j.swevo.2019.100594>, URL <https://www.sciencedirect.com/science/article/pii/S2210650219302317>.
- Waschneck, B., Reichstaller, A., Belzner, L., Altenmüller, T., Bauernhansl, T., Knapp, A., Kyek, A., 2018. Optimization of global production scheduling with deep reinforcement learning. *Procedia CIRP* 72, 1264–1269. <http://dx.doi.org/10.1016/j.procir.2018.03.212>.
- Wei, Y.Z., Zhao, M.Y., 2005. A reinforcement learning-based approach to dynamic job-shop scheduling. *Acta Automat. Sinica* 31 (5), 765–771.
- Wesendrup, K., Hellingrath, B., 2023. Post-prognostics demand management, production, spare parts and maintenance planning for a single-machine system using reinforcement learning. *Comput. Ind. Eng.* 179, 109216.
- Xiao, J., Wang, Z., He, J., Guohui, Y., 2023. A graph neural network based deep reinforcement learning algorithm for multi-agent leader-follower flocking. *Inform. Sci.* 641, 119074.
- Xili, C., XinChang, H., Hao, W.L., Tomohiro, M., 2010. Rule driven multi objective dynamic scheduling by data envelopment analysis and reinforcement learning. In: *IEEE International Conference on Automation and Logistics*. IEEE, pp. 396–401. <http://dx.doi.org/10.1109/ICAL.2010.5585316>.

- Xu, M., Mei, Y., Zhang, F., Zhang, M., 2024. Genetic programming and reinforcement learning on learning heuristics for dynamic scheduling: A preliminary comparison. *IEEE Comput. Intell. Mag.* 19 (2), 18–33. <http://dx.doi.org/10.1109/MCI.2024.3363970>.
- Yang, J., Nakhaei, A., Isele, D., Fujimura, K., Zha, H., 2020. CM3: Cooperative multi-goal multi-stage multi-agent reinforcement learning. In: *International Conference on Learning Representations*. [arXiv:1809.05188](https://arxiv.org/abs/1809.05188).
- Yu, R., Wang, X., Wang, R., Zhang, Y., An, B., Shi, Z.Y., Lai, H., 2020. Learning expensive coordination: An event-based deep RL approach. In: *International Conference on Learning Representations*.
- Yu, Q., Yang, H., Lin, K.Y., Li, L., 2021. A self-organized approach for scheduling semiconductor manufacturing systems. *J. Intell. Manuf.* 32 (3), 689–706. <http://dx.doi.org/10.1007/s10845-020-01678-8>.
- Zhang, Z., Luo, X., Liu, T., Xie, S., Wang, J., Wang, W., Li, Y., Peng, Y., 2019. Proximal policy optimization with mixed distributed training. In: *International Conference on Tools with Artificial Intelligence*. pp. 1452–1456. <http://dx.doi.org/10.1109/ICTAI.2019.00206>, [arXiv:1907.06479](https://arxiv.org/abs/1907.06479).
- Zhang, F., Mei, Y., Nguyen, S., Zhang, M., 2024. Survey on genetic programming and machine learning techniques for heuristic design in job shop scheduling. *IEEE Trans. Evol. Comput.* 28 (1), 147–167. <http://dx.doi.org/10.1109/TEVC.2023.3255246>.
- Zhang, R., Song, S., Wu, C., 2020. Robust scheduling of hot rolling production by local search enhanced ant colony optimization algorithm. *IEEE Trans. Ind. Inform.* 16 (4), 2809–2819. <http://dx.doi.org/10.1109/TII.2019.2944247>.
- Zhang, F., Yang, Q., An, D., 2022. A leader-following paradigm based deep reinforcement learning method for multi-agent cooperation games. *Neural Netw.* 156, 1–12. <http://dx.doi.org/10.1016/j.neunet.2022.09.012>, URL <https://www.sciencedirect.com/science/article/pii/S089360802200346X>.
- Zhang, K., Yang, Z., Başar, T., 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. In: *Handbook of Reinforcement Learning and Control*. pp. 321–384. http://dx.doi.org/10.1007/978-3-030-60990-0_12.